

Índice de Contenidos

1. Servidor HTTP con Node y Módulos Nativos
 - Módulos Nativos: fs y path
 - Servidor HTTP Básico
 - Ejemplo más completo
2. Introducción a Express
 - Ventajas de Express
 - Instalación e implementación
 - Servidor básico con Express
3. Ruteo Básico en Express, req/res
 - Métodos HTTP principales
 - Objetos Request y Response
4. Rutas y Middleware
 - Organización de rutas
 - Middleware en Express
5. Rutas Paramétricas y Middlewares Avanzados
 - Rutas Paramétricas
 - Validación de parámetros
 - Patrones de rutas
 - Middlewares Avanzados
 - Ejemplo de aplicación completa
6. Conclusión

1. Servidor HTTP con Node y Módulos Nativos

Módulos Nativos: fs y path

Node.js incluye varios módulos nativos que facilitan operaciones comunes:

Módulo fs (File System):

- Permite operaciones de lectura y escritura en el sistema de archivos
- Funciones principales:
 - `fs.readFile()`: Lee el contenido de un archivo
 - `fs.writeFile()`: Escribe contenido en un archivo
 - `fs.readdir()`: Lista el contenido de un directorio
 - `fs.mkdir()`: Crea un directorio nuevo

Módulo path:

- Ayuda a trabajar con rutas de archivos y directorios
- Funciones principales:
 - `path.join()`: Une segmentos de ruta de manera multiplataforma
 - `path.resolve()`: Resuelve una ruta absoluta
 - `path.basename()`: Devuelve el nombre del archivo de una ruta
 - `path.extname()`: Devuelve la extensión de un archivo

Servidor HTTP Básico

Para crear un servidor HTTP básico en Node.js, se utiliza el módulo nativo `http`:

```
// Importamos el módulo http
const http = require('http');

// Definimos el puerto
const PORT = 3000;

// Creamos el servidor
const server = http.createServer((req, res) => {
  // Configuramos la cabecera HTTP
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // Enviamos la respuesta
  res.end('Hola Mundo desde el servidor Node.js!\n');
});

// Ponemos el servidor a escuchar
server.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}/`);
});
```

```
});
```

Explicación:

1. Importamos el módulo `http`
2. Creamos un servidor con `http.createServer()` y definimos una función que se ejecuta cada vez que llega una petición
3. En esa función:
 - Configuramos la cabecera de respuesta (status 200 OK y tipo de contenido)
 - Enviamos la respuesta al cliente con `res.end()`
4. Iniciamos el servidor en el puerto especificado

Ejemplo más completo con rutas básicas y fs:

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = 3000;

const server = http.createServer((req, res) => {
  // Obtenemos la URL solicitada
  const url = req.url;

  // Ruta básica
  if (url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Página de inicio</h1>');
  }

  // Ruta /about
  else if (url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Acerca de nosotros</h1>');
  }

  // Servir un archivo
  else if (url === '/archivo') {
    const filePath = path.join(__dirname, 'ejemplo.txt');

    fs.readFile(filePath, (err, content) => {
      if (err) {
        res.writeHead(500);
        res.end('Error en el servidor');
        return;
      }
    })
  }
});
```

```
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        res.end(content);
    });
}
// Ruta no encontrada
else {
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end('<h1>404 - Página no encontrada</h1>');
}
});

server.listen(PORT, () => {
    console.log(`Servidor corriendo en http://localhost:${PORT}/`);

});
```

2. Introducción a Express

Express es un framework web minimalista y flexible para Node.js que proporciona un conjunto robusto de características para aplicaciones web y móviles.

Ventajas de Express sobre HTTP nativo:

- Manejo simplificado de rutas
- Sistema de middleware integrado
- Integración fácil con motores de plantillas
- Mejor manejo de errores
- Más fácil de escalar y mantener

Instalación e implementación básica:

```
# Crear directorio del proyecto
mkdir mi-app-express
cd mi-app-express

# Inicializar proyecto Node.js
npm init -y
```

```
# Instalar Express
```

```
npm install express
```

Servidor básico con Express:

```
// Importamos Express
```

```
const express = require('express');
```

```
// Creamos una instancia de la aplicación Express
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Definimos una ruta para la página principal
```

```
app.get('/', (req, res) => {
```

```
  res.send('¡Hola Mundo desde Express!');
```

```
});
```

```
// Iniciamos el servidor
```

```
app.listen(PORT, () => {
```

```
  console.log(`Servidor Express ejecutándose en http://localhost:${PORT}`);
```

```
});
```

3. Ruteo Básico en Express, req/res

Express simplifica enormemente el manejo de rutas HTTP:

Métodos HTTP principales:

```
const express = require('express');
```

```
const app = express();
```

```
// GET - Obtener recursos
```

```
app.get('/usuarios', (req, res) => {
```

```

    res.send('Lista de usuarios');
  });

// POST - Crear recursos
app.post('/usuarios', (req, res) => {
  res.send('Usuario creado');
});

// PUT - Actualizar recursos
app.put('/usuarios/:id', (req, res) => {
  res.send(`Usuario ${req.params.id} actualizado`);
});

// DELETE - Eliminar recursos
app.delete('/usuarios/:id', (req, res) => {
  res.send(`Usuario ${req.params.id} eliminado`);
});

```

```
app.listen(3000);
```

Objetos Request (req) y Response (res):

Objeto Request (req):

- `req.params`: Parámetros de ruta (URL)
- `req.query`: Parámetros de consulta (query string)
- `req.body`: Datos enviados en el cuerpo de la petición (requiere middleware)
- `req.headers`: Cabeceras HTTP
- `req.cookies`: Cookies (requiere middleware)
- `req.ip`: Dirección IP del cliente
- `req.path`: Ruta solicitada
- `req.method`: Método HTTP utilizado

Objeto Response (res):

- `res.send()`: Envía una respuesta
- `res.json()`: Envía una respuesta en formato JSON
- `res.status()`: Establece el código de estado HTTP

- `res.redirect()`: Redirecciona a otra URL
- `res.render()`: Renderiza una vista/plantilla
- `res.download()`: Inicia la descarga de un archivo
- `res.sendFile()`: Envía un archivo
- `res.cookie()`: Establece una cookie

Ejemplo:

```
app.get('/producto', (req, res) => {
  // Obtener parámetros de query string (?id=123&categoria=electrónica)
  const id = req.query.id;
  const categoria = req.query.categoria;

  res.status(200).json({
    id: id,
    categoria: categoria,
    mensaje: 'Datos recibidos correctamente'
  });
});
```

4. Rutas y Middleware

Organización de rutas:

Express permite organizar rutas en archivos separados usando

`express.Router()`:

```
// archivo: routes/usuarios.js
const express = require('express');
const router = express.Router();

// Estas rutas se montarán en /usuarios/...
router.get('/', (req, res) => {
  res.send('Lista de todos los usuarios');
});
```

```
router.get('/:id', (req, res) => {  
  res.send(`Detalles del usuario ${req.params.id}`);  
});
```

```
router.post('/', (req, res) => {  
  res.send('Crear nuevo usuario');  
});
```

```
module.exports = router;
```

```
// archivo: app.js  
const express = require('express');  
const app = express();  
const usuariosRouter = require('./routes/usuarios');
```

```
// Montamos las rutas de usuarios en /usuarios  
app.use('/usuarios', usuariosRouter);
```

```
app.listen(3000);
```

Middleware:

El middleware son funciones que tienen acceso al objeto de solicitud (req), al objeto de respuesta (res), y a la siguiente función middleware en el ciclo de solicitud/respuesta.

Funciones principales:

- Ejecutar código
- Modificar los objetos req y res
- Finalizar el ciclo de solicitud/respuesta
- Llamar al siguiente middleware

Tipos de middleware:

1. Middleware de nivel de aplicación:

```
// Middleware que se ejecuta en todas las solicitudes
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date()}`);
  next(); // Llama al siguiente middleware
```

2. `});`

3. Middleware de nivel de ruta:

```
app.get('/usuario/:id', (req, res, next) => {
  // Verificar si el usuario existe
  if (usuarioExiste(req.params.id)) {
    next(); // Continuar a la siguiente función
  } else {
    res.status(404).send('Usuario no encontrado');
  }
}, (req, res) => {
  // Esta función solo se ejecuta si next() fue llamado
  res.send(`Datos del usuario ${req.params.id}`);
```

4. `});`

5. Middleware de manejo de errores:

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send(';Algo salió mal!');
```

6. `});`

7. Middleware de terceros:

```
const express = require('express');
const morgan = require('morgan'); // Middleware de logging
const bodyParser = require('body-parser'); // Middleware para parsear cuerpo de peticiones
```

```
const app = express();
```

```
// Usar middlewares
```

```
app.use(morgan('dev')); // Log de solicitudes
app.use(bodyParser.json()); // Parsear solicitudes con JSON
```

```
8. app.use(bodyParser.urlencoded({ extended: true })); // Parsear formularios
```

5. Rutas Paramétricas y Middlewares Avanzados

Rutas Paramétricas:

Las rutas paramétricas permiten manejar URLs dinámicas:

```
// Parámetro único
app.get('/usuario/:id', (req, res) => {
  res.send(`Usuario con ID: ${req.params.id}`);
});

// Múltiples parámetros
app.get('/productos/:categoria/:id', (req, res) => {
  const categoria = req.params.categoria;
  const id = req.params.id;
  res.send(`Producto ${id} de la categoría ${categoria}`);
});

// Parámetros opcionales
app.get('/articulos/:año?/:mes?/:dia?', (req, res) => {
  const año = req.params.año || 'todos los años';
  const mes = req.params.mes || 'todos los meses';
  const dia = req.params.dia || 'todos los días';

  res.send(`Artículos de: ${dia} de ${mes} de ${año}`);
});
```

Validación de parámetros:

```
// Middleware para validar un ID
function validarId(req, res, next) {
  const id = req.params.id;

  // Validar que sea un número
  if (isNaN(id)) {
    return res.status(400).send('ID inválido: debe ser un número');
  }

  // Convertir a número y guardarlo en req para uso posterior
  req.idValidado = parseInt(id);
  next();
}

// Uso del middleware de validación
app.get('/usuario/:id', validarId, (req, res) => {
  // Aquí req.idValidado ya está disponible y validado
  res.send(`Usuario válido con ID: ${req.idValidado}`);
});
```

Patrones de rutas con expresiones regulares:

```
// Coincide con rutas que contienen 'abc'
app.get(/abc/, (req, res) => {
  res.send('Ruta contiene "abc"');
});

// Coincide con rutas que terminan en '.html'
app.get(/.*\.html$/, (req, res) => {
  res.send('Archivo HTML solicitado');
});
```

Middleware Avanzados:

Middleware condicional:

```
// Middleware que solo se ejecuta si existe una condición
function soloAdmin(req, res, next) {
  if (req.query.rol === 'admin') {
    next(); // Usuario es admin, continuar
  } else {
    res.status(403).send('Acceso denegado: solo administradores');
  }
}

// Aplicar el middleware solo a ciertas rutas
app.get('/admin-panel', soloAdmin, (req, res) => {
  res.send('Panel de administración');
});

// Ruta normal sin restricción
app.get('/public', (req, res) => {
  res.send('Área pública');
});
```

Middleware para CORS (Cross-Origin Resource Sharing):

```
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type, Accept');
  next();
});
```

Manejo de archivos estáticos:

```
// Servir archivos estáticos desde la carpeta 'public'
app.use(express.static('public'));

// Con prefijo de ruta
```

```
app.use('/assets', express.static('public'));
```

Ejemplo de aplicación completa:

```
const express = require('express');
const path = require('path');
const fs = require('fs');
const morgan = require('morgan'); // npm install morgan

const app = express();
const PORT = 3000;

// Middlewares
app.use(morgan('dev')); // Logging
app.use(express.json()); // Parsear JSON
app.use(express.urlencoded({ extended: true })); // Parsear formularios
app.use(express.static('public')); // Archivos estáticos

// Middleware personalizado para registro de tiempo
app.use((req, res, next) => {
  req.tiempoInicio = Date.now();
  next();
});

// Middleware para finalizar medición de tiempo
app.use((req, res, next) => {
  const oldSend = res.send;
  res.send = function(data) {
    console.log(`Tiempo de respuesta: ${Date.now() - req.tiempoInicio}ms`);
    oldSend.apply(res, arguments);
  };
  next();
});

// Rutas
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'views', 'index.html'));
});

// Rutas paramétricas
app.get('/api/producto/:id', (req, res) => {
  const id = req.params.id;
  // Simulamos búsqueda en base de datos
  const producto = {
    id: id,
    nombre: `Producto ${id}`,
    precio: Math.floor(Math.random() * 1000)
  };
  res.json(producto);
});
```

```

// Router para API
const apiRouter = express.Router();

apiRouter.get('/usuarios', (req, res) => {
  res.json([
    { id: 1, nombre: 'Ana' },
    { id: 2, nombre: 'Juan' },
    { id: 3, nombre: 'Carlos' }
  ]);
});

apiRouter.post('/usuarios', (req, res) => {
  // Accedemos a los datos enviados gracias al middleware express.json()
  const nuevoUsuario = req.body;
  console.log('Datos recibidos:', nuevoUsuario);

  // Simulamos creación en base de datos
  nuevoUsuario.id = Date.now();

  res.status(201).json({
    mensaje: 'Usuario creado correctamente',
    usuario: nuevoUsuario
  });
});

// Montamos el router
app.use('/api', apiRouter);

// Middleware para manejo de errores 404
app.use((req, res, next) => {
  res.status(404).sendFile(path.join(__dirname, 'views', '404.html'));
});

// Middleware para manejo de errores generales
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('¡Algo salió mal en el servidor!');
});

// Iniciamos el servidor
app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});

```

Conclusión

Node.js con Express proporciona una plataforma poderosa y flexible para crear aplicaciones web y APIs. La combinación de los módulos nativos de Node con las características avanzadas de Express permite desarrollar servidores robustos con un código más limpio y mantenible.

La estructura de middleware de Express es particularmente valiosa, ya que permite separar la lógica en componentes reutilizables y mantener un flujo de control claro para las solicitudes HTTP.

Nota: Este material está diseñado como recurso educativo. Se recomienda practicar con los ejemplos proporcionados para afianzar los conocimientos.